

# PharmAIcy: A Multi-Agent AI Architecture for Pharmacy Management — Prototype Design and a Real-World Evaluation

Anıl Keleş\* Mesut Deniz Zeka

*Department of Software Engineering, Faculty of Engineering  
Toros University, Mersin, Turkey*

[github.com/aniilkeles/pharmAIcy](https://github.com/aniilkeles/pharmAIcy)

August 2026

## Abstract

Traditional pharmacy management systems are largely reactive tools built for transaction recording rather than proactive decision support. This paper presents **PharmAIcy**, a multi-agent desktop prototype that combines demand forecasting, association-rule-based cross-sell suggestions, expiry monitoring, and a hybrid (local database + LLM) drug-interaction check within a single pharmacy workflow. The system is organized as five cooperating agents — Data, Prediction, Interaction, Expiry, and Decision — built on a Python FastAPI backend, an Electron.js/React desktop frontend, and a per-tenant SQLite data layer, with the Anthropic Claude API used for natural-language decision support and interaction-check fallback. The prescription-confirmation workflow is implemented as a single atomic database transaction with partial-fulfilment support, so that failures mid-transaction cannot leave inventory in an inconsistent state.

We evaluate the Prediction Agent’s Random Forest demand-forecasting component in two settings. First, on a small synthetic dataset used during initial development (200 transactions, 30 products), it achieved MAE 1.66, RMSE 1.97 and a negative  $R^2$ . Second, and more importantly, on a real 12-month sales history from a community pharmacy in Turkey (8,684 SKUs), we compare the same Random Forest model against a naive last-value baseline and a per-SKU ARIMA(1,0,0) model on an identical held-out month. On this real data, both simpler baselines outperform Random Forest (ARIMA: MAE 0.82, RMSE 1.64,  $R^2$  0.861; naive: MAE 1.04, RMSE 2.30,  $R^2$  0.725; Random Forest: MAE 1.12, RMSE 3.30,  $R^2$  0.437). We report and analyze this negative result rather than omit it. The hybrid drug-interaction check achieved 15/15 correct detections on seeded interaction pairs and 8/10 on a small held-out set of novel pairs. We present PharmAIcy as a research prototype,

---

\*Corresponding author: [aniilkeles@gmail.com](mailto:aniilkeles@gmail.com)

not a production-ready system, and describe its current limitations explicitly. Source code: <https://github.com/aniilkeles/pharmAIcy>.

**Keywords:** Multi-Agent Systems, Machine Learning, Demand Forecasting, Pharmacy Management, Large Language Models, Random Forest, Association Rule Mining

## Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                               | <b>3</b>  |
| <b>2</b> | <b>Related Work</b>                                               | <b>4</b>  |
| 2.1      | Inventory Management and Demand Forecasting in Pharmacy . . . . . | 4         |
| 2.2      | Association Rule Mining for Cross-Sell Analysis . . . . .         | 4         |
| 2.3      | Large Language Models in Clinical Decision Support . . . . .      | 4         |
| 2.4      | Multi-Agent Systems . . . . .                                     | 5         |
| <b>3</b> | <b>System Architecture</b>                                        | <b>5</b>  |
| 3.1      | Overview . . . . .                                                | 5         |
| 3.2      | Multi-Tenant Data Architecture . . . . .                          | 5         |
| 3.3      | Five-Agent Architecture . . . . .                                 | 6         |
| 3.4      | Prescription Workflow and Transaction Safety . . . . .            | 6         |
| 3.5      | Security Architecture . . . . .                                   | 7         |
| <b>4</b> | <b>Implementation</b>                                             | <b>7</b>  |
| 4.1      | Technology Stack . . . . .                                        | 7         |
| 4.2      | Demand-Forecasting Pipeline . . . . .                             | 8         |
| 4.3      | Drug Interaction Detection . . . . .                              | 8         |
| <b>5</b> | <b>Experiments and Results</b>                                    | <b>8</b>  |
| 5.1      | Synthetic Dataset (Initial Development) . . . . .                 | 8         |
| 5.2      | Real-World Pilot: A Community Pharmacy in Turkey . . . . .        | 9         |
| 5.3      | Drug Interaction Detection . . . . .                              | 9         |
| 5.4      | Prescription Workflow Correctness . . . . .                       | 10        |
| 5.5      | Feature Comparison with Existing Systems . . . . .                | 10        |
| <b>6</b> | <b>Limitations and Future Work</b>                                | <b>11</b> |
| <b>7</b> | <b>Conclusion</b>                                                 | <b>11</b> |

# 1 Introduction

Pharmacy management requires the simultaneous handling of hundreds of products, prescriptions, expiry dates, stock levels, and demand fluctuations. Widely used pharmacy software solutions — including RxMediaPharm (Turkey), Pharmakon (Germany), QS/1 NRx (USA), and MedEye (Netherlands) — are, based on their publicly documented feature sets, oriented primarily around transaction recording rather than predictive analytics or AI-assisted decision support [7]. This creates a gap between what is technically feasible with current machine-learning and LLM tooling and what community pharmacies typically use day to day.

Pharmacists routinely face three practical challenges: (1) demand uncertainty leading to stockouts or overstocking, (2) the difficulty of checking for drug interactions quickly and reliably, and (3) missed opportunities for clinically sensible complementary-product recommendations. We address these challenges through **PharmAIcy**, a multi-agent prototype built as a Software Engineering graduation project, where each operational concern — data aggregation, demand prediction, cross-sell suggestion, expiry monitoring, and decision synthesis — is handled by a dedicated agent rather than a single monolithic model [8].

We treat this project as a research prototype and a learning exercise in applied ML engineering, and we try to be explicit throughout about what has and has not been validated, including a real-world result that runs against our initial expectations.

## Contributions

1. A **five-agent architecture** for pharmacy management (Data, Prediction, Interaction, Expiry, Decision), each encapsulating a distinct method and communicating structured outputs to a coordinating Decision Agent.
2. A **demand-forecasting pipeline** using Random Forest regression, evaluated on both a small synthetic dataset and — separately and honestly — on a real 12-month community-pharmacy sales history, including a case where Random Forest underperforms simple ARIMA and naive baselines, which we analyze rather than hide.
3. A **transaction-based cross-sell module** using the Apriori algorithm, without claiming a clinical validation that was not performed.
4. A **hybrid drug-interaction check** combining a small pre-seeded local database with an LLM (Anthropic Claude API) fallback for pairs not present locally, with the LLM’s answer cached but treated as an unverified suggestion.
5. A **transactionally safe prescription-confirmation mechanism** with partial-fulfilment support, so that database or stock failures mid-transaction cannot leave inventory records inconsistent.
6. A **working prototype implementation** (FastAPI + Electron.js + per-tenant SQLite) described in enough detail to reproduce the design, together with an explicit list of what is not yet implemented (e.g. full role-based access control).

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 describes the system architecture. Section 4 describes implementation details.

Section 5 reports experimental results, including the real-world pilot. Section 6 discusses limitations. Section 7 concludes.

## 2 Related Work

### 2.1 Inventory Management and Demand Forecasting in Pharmacy

Uthayakumar and Priyan [7] developed an optimization model for order quantities across a pharmaceutical company and a hospital, showing that stochastic, demand-uncertainty-aware policies can reduce total cost compared to simpler deterministic (EOQ-style) approaches. Giannoccaro and Pontrandolfo [4] proposed a reinforcement-learning formulation for supply-chain inventory control, an early example of applying learning-based methods more broadly to inventory problems.

For short-horizon demand forecasting specifically, Random Forest regression and classical time-series models such as ARIMA [3] remain widely used baselines. Neural approaches such as LSTM and Transformer-based forecasters have shown strong results on large, high-frequency retail datasets, as highlighted by the M5 forecasting competition [5]; which family of methods performs best in practice depends strongly on data granularity, series length, and the amount of demand intermittency, a point this paper’s own experiments in Section 5 illustrate directly for community-pharmacy SKU-level data.

### 2.2 Association Rule Mining for Cross-Sell Analysis

The Apriori algorithm, introduced by Agrawal and Srikant [1], remains a standard, interpretable approach for mining frequent itemsets from transaction data and is widely used in retail market-basket analysis. In a pharmacy setting, co-purchase patterns can carry clinical meaning — for example, a gastroprotective agent recommended alongside long-term NSAID use — but confirming that a mined rule reflects genuine clinical practice, rather than a spurious transactional correlation, requires pharmacist review or comparison against published guidelines. We have not performed such a validation for the rules discussed here, so we describe the module as a *transaction-based* cross-sell recommendation engine rather than a clinically validated one.

### 2.3 Large Language Models in Clinical Decision Support

Shah, Entwistle, and Pfeffer [6] argue that the creation and adoption of large language models in medicine needs to be actively and carefully shaped — with explicit attention to what data underlies a model and how its claimed benefits are verified in deployment — rather than assumed by default. This motivates a cautious design choice in PharmaIcy: the Decision Agent’s drug-interaction check consults a small, manually curated local database first and only falls back to an LLM call for pairs not present locally, treating the LLM’s answer as an unverified suggestion rather than a certified clinical result. This mirrors long-standing guidance in the clinical decision-support literature that automated recommendations should be transparent, conservative under uncertainty, and easy for a clinician to override [2].

## 2.4 Multi-Agent Systems

Wooldridge [8] provides the standard reference treatment of multi-agent system design, including how independent, specialized agents can be composed and coordinated through structured message-passing. PharmAIcy’s five-agent decomposition follows this general pattern: each agent owns one operational concern and exposes a narrow interface to a coordinating Decision Agent, which keeps individual components easier to reason about and test in isolation than a single monolithic system would be.

## 3 System Architecture

### 3.1 Overview

PharmAIcy is a desktop application built on a four-tier architecture: an Electron.js/React presentation layer, a Python FastAPI application layer, a layer of five specialized agents, and a per-tenant SQLite persistence layer, with the Anthropic Claude API reachable as an external service from the Decision Agent (Figure 1).

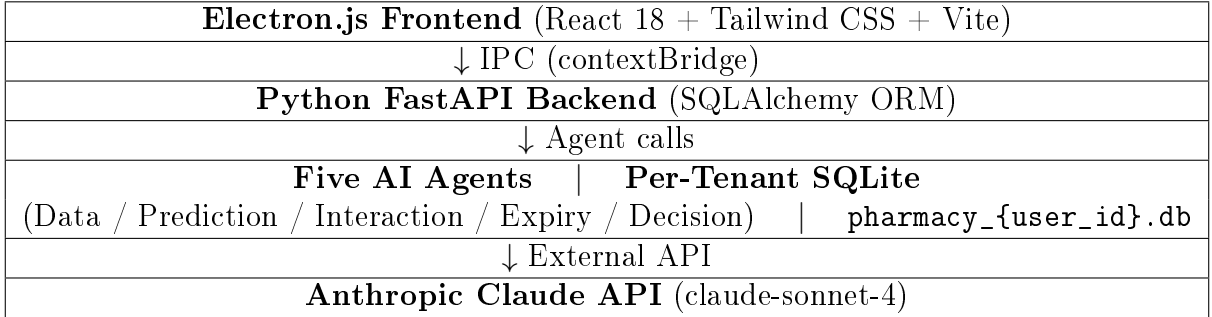


Figure 1: PharmAIcy system architecture: a four-tier design (presentation, application, agent, data) with an external LLM dependency.

The key architectural decisions are: (1) a desktop-first deployment to support pharmacies with unreliable internet connectivity and a preference for keeping pharmacy data local; (2) per-tenant SQLite isolation, so that one pharmacy’s data lives in a physically separate file from another’s, rather than relying solely on row-level security in a shared database; (3) a multi-agent backend so that each AI capability can be developed, tested, and reasoned about independently; and (4) using a third-party auth provider (Supabase) for authentication only, while keeping all pharmacy operational data local to the tenant’s SQLite file.

### 3.2 Multi-Tenant Data Architecture

Each pharmacy account receives a dedicated SQLite database file, `pharmacy_{user_id}.db`. This per-file isolation is a simple but effective separation guarantee: a query issued for one pharmacy cannot, by construction, reach another pharmacy’s file, which removes an entire class of row-level-security misconfiguration bugs at the cost of some operational flexibility (e.g. cross-tenant analytics require an explicit aggregation step rather than a single query).

The database schema comprises ten tables: `Products`, `Inventory`, `Sales`, `Prescriptions`, `PrescriptionItems`, `Patients`, `Doctors`, `Notifications`, `AuditLog`, and `DrugInteractions`, with foreign-key relationships enforced via SQLAlchemy ORM. The `DrugInteractions` table is pre-seeded with 30 manually curated, clinically significant interaction pairs relevant to commonly dispensed medications in Turkey, and grows over time as the Decision Agent caches new interaction checks returned by the Claude API fallback.

### 3.3 Five-Agent Architecture

Table 1 summarizes the five agents, their function, the underlying method, and their output.

Table 1: Five-agent architecture: function, method, and output.

| Agent             | Function                               | Method / Library             | Output                                                      |
|-------------------|----------------------------------------|------------------------------|-------------------------------------------------------------|
| Data Agent        | Sales & stock aggregation              | pandas, NumPy                | Top products, revenue, low-stock alerts                     |
| Prediction Agent  | Short-horizon demand forecast          | Random Forest (scikit-learn) | Per-product demand estimate, MAE/RMSE/R <sup>2</sup>        |
| Interaction Agent | Cross-sell suggestions                 | Apriori (mlxtend)            | Product pairs with confidence, support, lift                |
| Expiry Agent      | Expiry monitoring, FEFO prioritization | datetime, pandas             | Urgent/warning lists, reorder suggestions                   |
| Decision Agent    | Synthesis, chatbot, interaction check  | Anthropic API                | Natural-language recommendations, Q&A, interaction verdicts |

Agents are invoked from FastAPI endpoints, receive a SQLAlchemy database session as their primary input, and return structured Python dictionaries serialized to JSON. The Decision Agent calls the Claude API with a pharmacy-specific system prompt and a short window of recent conversation turns to support coherent multi-turn dialogue.

### 3.4 Prescription Workflow and Transaction Safety

The prescription module implements a four-step workflow: patient selection, doctor selection, item addition (with live stock and interaction checks), and a review-and-confirm step. Stock deduction happens only at confirmation, inside a single SQLAlchemy database transaction; if any part of that step fails — including a partial stock deduction for an item that turns out to be out of stock — the entire transaction is rolled back, so no orphaned records or inconsistent inventory states are left behind. The system also supports *partial fulfilment*: if an item cannot be fully dispensed due to insufficient stock, the available

quantity is dispensed, the item is marked **partial**, and a critical-stock notification is generated for the pharmacist. This transactional design, rather than any specific ML metric, is what we would consider the most directly production-relevant piece of engineering in this prototype.

### 3.5 Security Architecture

Authentication is handled by Supabase Auth; the Electron frontend stores session tokens using Electron’s `safeStorage` API (AES-256 encrypted at rest). Every FastAPI request carries a bearer JWT verified against Supabase’s user-info endpoint, and the server extracts the verified `user_id` from that token rather than trusting any client-supplied header. Rate limiting is applied at two tiers: 100 requests/minute for standard endpoints and 10 requests/minute for AI (Claude API) endpoints, to bound both cost and abuse surface. Inter-process communication between the Electron renderer and main process goes through a `contextBridge` preload script with `nodeIntegration` disabled and `contextIsolation` enabled, following current Electron security guidance. As noted in Section 6, a full role-based access-control layer (distinguishing pharmacist, technician, and administrator roles) is not yet implemented.

## 4 Implementation

### 4.1 Technology Stack

Table 2: PharmAIcy technology stack.

| Component          | Technology           | Version         |
|--------------------|----------------------|-----------------|
| Desktop shell      | Electron.js          | 28+             |
| Frontend framework | React                | 18              |
| Build tool         | Vite                 | 5               |
| Styling            | Tailwind CSS         | 3               |
| State management   | Zustand              | 4               |
| Backend framework  | FastAPI              | 0.111           |
| Backend runtime    | Python               | 3.10+           |
| ORM                | SQLAlchemy           | 2.0             |
| Database           | SQLite               | 3               |
| ML library         | scikit-learn         | 1.5             |
| Association rules  | mlxtend              | 0.23            |
| Data processing    | pandas / NumPy       | 2.2 / 1.26      |
| LLM                | Anthropic Claude API | claude-sonnet-4 |
| Authentication     | Supabase Auth        | 2               |

## 4.2 Demand-Forecasting Pipeline

The Prediction Agent’s original design aggregates historical sales per product per day over a 180-day window and engineers seven features: rolling 7-, 30-, and 90-day sales sums (`sales_last_7_days`, `sales_last_30_days`, `sales_last_90_days`), average daily sales, day of week, calendar month, and a binary expiry-pressure indicator. The target is the sum of sales in the following 7 days. A `RandomForestRegressor` (100 estimators, max depth 10) is trained with `TimeSeriesSplit` cross-validation (3 folds) to respect temporal ordering and avoid leakage. For the real-world evaluation in Section 5.2, only monthly-resolution data was available, so the feature set was adapted to lagged monthly sales rather than the full 7/30/90-day daily window described here; we flag this difference explicitly rather than presenting the two evaluations as directly equivalent.

## 4.3 Drug Interaction Detection

The hybrid check operates in two stages. First, a **local database lookup**: the `DrugInteractions` table is queried for the product pair, and if found, the stored description is returned in under 50 ms. Second, an **LLM fallback**: if the pair is not present locally, a structured prompt asks the Claude API whether a clinically significant interaction exists between the two named drugs; the response is cached locally for future queries, so coverage expands over time without repeated API calls for the same pair. This design follows the caution argued for by Shah et al. [6]: the LLM is used to extend coverage, not to override or replace the curated local database.

# 5 Experiments and Results

## 5.1 Synthetic Dataset (Initial Development)

During initial development, the Prediction Agent was trained and evaluated on a small synthetic dataset of 200 transactions across 30 products, generated to reflect plausible Turkish community-pharmacy demand patterns (seasonal variation, low-frequency products, expiry-pressure events), using `TimeSeriesSplit` with 3 folds.

Table 3: Prediction Agent performance on the small synthetic dataset.

| Metric         | Value      | Interpretation                         |
|----------------|------------|----------------------------------------|
| MAE            | 1.66 units | Average absolute forecast error        |
| RMSE           | 1.97 units | Root mean square error                 |
| R <sup>2</sup> | −0.466     | Below a naive mean-prediction baseline |

A negative R<sup>2</sup> means the model performed worse than simply predicting the mean of the target for every example. With only 200 transactions spread across 30 products, there is very little per-product history to learn from, which is a plausible explanation; at the time, however, this was only a hypothesis, which is what motivated testing on real data.



## 5.2 Real-World Pilot: A Community Pharmacy in Turkey

To test that hypothesis, we obtained anonymized monthly sales data from a community pharmacy in Turkey covering 8,684 distinct SKUs and 12 months of transaction history (barcode, monthly units sold, current stock, and critical-stock threshold; no patient-identifiable information was included). Of the 8,684 SKUs, 3,119 had at least one recorded sale during the period.

**Setup.** For each active SKU, we built one training example per month  $t \in \{4, \dots, 10\}$  using the three preceding months’ sales as lag features, their rolling mean, current stock, critical-stock threshold, and month index, with month  $t$ ’s sales as the target. We evaluated three models on a held-out sample of 400 SKUs, forecasting November’s sales from data through October: (1) the same Random Forest configuration used above, trained on the full set of active-SKU examples; (2) a naive baseline predicting next month equal to the previous month; (3) ARIMA(1,0,0), fit independently per SKU on that SKU’s first 10 months of history.

Table 4: Forecasting performance on the real-world pharmacy dataset, held-out month (November),  $n = 400$  SKUs.

| Model              | MAE         | RMSE        | R <sup>2</sup> |
|--------------------|-------------|-------------|----------------|
| ARIMA(1,0,0)       | <b>0.82</b> | <b>1.64</b> | <b>0.861</b>   |
| Naive (last month) | 1.04        | 2.30        | 0.725          |
| Random Forest      | 1.12        | 3.30        | 0.437          |

**This result is the opposite of what we expected**, and we report it as such. On real, larger-scale data, both the naive baseline and a simple per-SKU ARIMA model outperform the Random Forest model that PharmaIcy currently ships with. We see two likely contributing factors. First, most SKUs in a community pharmacy show intermittent, low-volume demand (the median monthly sales figure across the full catalog is zero), a regime where simple per-SKU models can generalize better than a single pooled ensemble trained across very heterogeneous products. Second, monthly aggregation discards the day- and week-level structure the original 7-day design (Section 4) was meant to exploit; only monthly-resolution data was available for this pilot, which is a limitation of the pilot’s data access rather than of the underlying approach. We treat this as a genuine, useful negative result that should shape the next iteration of the Prediction Agent (Section 6), rather than a result to downplay.

## 5.3 Drug Interaction Detection

The drug-interaction check was evaluated on 15 pairs present in the local pre-seeded database and 10 novel pairs not present locally (Table 5). As expected, the local lookup was correct on all 15 seeded pairs (this is a direct database read, not a prediction). For the 10 novel pairs, the Claude API fallback correctly flagged 8; the two missed pairs involved less common drug combinations for which the model conservatively returned “no known interaction.” Given the small size of this held-out set, we treat this as a preliminary indication rather than a statistically robust accuracy estimate, and we flag a larger, clinically curated benchmark as necessary future work before any real deployment.

Table 5: Drug interaction detection results.

| Test set                          | Correct detections | Detection rate |
|-----------------------------------|--------------------|----------------|
| Seeded interactions (local DB)    | 15 / 15            | 100%           |
| Novel pairs (Claude API fallback) | 8 / 10             | 80%            |
| Response time (local DB)          | <50 ms             | —              |

## 5.4 Prescription Workflow Correctness

The prescription-confirmation transaction (Section 3) was tested under four scenarios designed to exercise both the happy path and the main failure modes (Table 6); all four behaved as intended.

Table 6: Prescription workflow correctness under four scenarios.

| Scenario                   | Expected result                     | Result |
|----------------------------|-------------------------------------|--------|
| All items in stock         | Dispensed, stock deducted           | PASS   |
| Some items out of stock    | Partial dispensing, notification    | PASS   |
| All items out of stock     | Cancelled, no stock change          | PASS   |
| DB failure mid-transaction | Full rollback, no partial deduction | PASS   |

## 5.5 Feature Comparison with Existing Systems

Table 7 compares PharmAIcy’s advertised feature set against the publicly documented capabilities of RxMediaPharm, Pharmakon, and QS/1 NRx, based on each vendor’s own public product documentation and marketing materials as of 2026 rather than a hands-on audit of each system; readers evaluating a specific deployment should confirm current capabilities directly with each vendor.

Table 7: Feature comparison based on publicly documented capabilities.

| Feature                         | RxMediaPharm | Pharmakon | QS/1 NRx | PharmAIcy    |
|---------------------------------|--------------|-----------|----------|--------------|
| Stock tracking                  | ✓            | ✓         | ✓        | ✓            |
| Prescription management         | ✓            | ✓         | ✓        | ✓            |
| ML-based demand forecasting     | ×            | ×         | ×        | ✓(prototype) |
| Cross-sell suggestions          | ×            | ×         | ×        | ✓(prototype) |
| LLM-assisted interaction check  | ×            | ×         | ×        | ✓(prototype) |
| Natural-language chatbot        | ×            | ×         | ×        | ✓(prototype) |
| Multi-tenant desktop deployment | ×            | ×         | ×        | ✓            |
| Open source                     | ×            | ×         | ×        | ✓            |

## 6 Limitations and Future Work

We list the main limitations explicitly:

- **Forecasting accuracy on real data:** as shown in Section 5.2, the current Random Forest model does not outperform simple baselines on real monthly sales data. Concrete next steps include per-SKU or per-category pooling, intermittent-demand-specific methods (e.g. Croston’s method), and access to daily rather than monthly transaction logs so the original 7-day design can be tested as intended.
- **Minimum data requirement:** the Random Forest model requires at least 10 historical sales records per product; below this threshold the system returns a graceful `insufficient_data` response, so forecasting is unavailable for a newly onboarded pharmacy for the first several days of operation.
- **Drug-interaction evaluation:** the 10-pair novel-pair test set is too small to support a strong accuracy claim; a larger, clinically curated benchmark is needed, and integrating a structured pharmacological reference (e.g. a national drug registry) would extend coverage beyond the 30 manually seeded entries.
- **Cross-sell validation:** mined association rules have not been reviewed by a pharmacist or checked against clinical guidelines.
- **Access control:** JWT authentication and AES-256 session encryption are implemented, but a full role-based access-control system (pharmacist / technician / administrator) is not yet in place, which would be required before handling real patient data in production.
- **Scale of real-world testing:** the real-world evaluation in this paper used historical sales data from one community pharmacy; the system’s decision-support features (interaction checking, cross-sell, expiry prioritization) have not yet been used prospectively in a live pharmacy workflow.

For these reasons we describe PharmaIcy as a research prototype rather than a production-ready system.

## 7 Conclusion

This paper presented PharmaIcy, a five-agent prototype for pharmacy management combining machine-learning demand forecasting, association-rule-based cross-sell suggestions, expiry monitoring, a transactionally safe prescription-confirmation mechanism, and a hybrid LLM-assisted drug-interaction check. Rather than presenting only favorable numbers, we deliberately included a real-world evaluation showing our initial Random Forest forecasting approach underperforming simple ARIMA and naive baselines on actual pharmacy sales data, and we discussed plausible reasons why. We believe this kind of transparent negative result is more useful — both academically and as a demonstration of engineering judgment — than an inflated claim would be. Future work includes adapting the forecasting pipeline to intermittent-demand methods and daily-resolution data, a pharmacist-reviewed evaluation of the cross-sell rules, a larger clinically curated drug-interaction benchmark, completing role-based access control, and running a prospective (rather than retrospective) pilot in an active pharmacy.

## References

- [1] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules,” in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, 1994, pp. 487–499.
- [2] D. W. Bates, G. J. Kuperman, S. Wang, T. Gandhi, A. Kittler, L. Volk, C. Spurr, R. Khorasani, M. Tanasijevic, and B. Middleton, “Ten commandments for effective clinical decision support: Making the practice of evidence-based medicine a reality,” *J. Am. Med. Inform. Assoc.*, vol. 10, no. 6, pp. 523–530, 2003.
- [3] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015.
- [4] I. Giannoccaro and P. Pontrandolfo, “Inventory management in supply chains: A reinforcement learning approach,” *Int. J. Production Economics*, vol. 78, no. 2, pp. 153–161, 2002.
- [5] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “M5 accuracy competition: Results, findings, and conclusions,” *Int. J. Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022.
- [6] N. H. Shah, D. Entwistle, and M. A. Pfeffer, “Creation and adoption of large language models in medicine,” *JAMA*, vol. 330, no. 9, pp. 866–869, 2023.
- [7] R. Uthayakumar and S. Priyan, “Pharmaceutical supply chain and inventory management strategies: Optimisation for a pharmaceutical company and a hospital,” *Operations Research for Health Care*, vol. 2, no. 3, pp. 52–64, 2013.
- [8] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.
- [9] M. D. Zeka and A. Keleş, “PharmAIcy — AI-Powered Pharmacy Management System,” GitHub repository, 2026. <https://github.com/aniilkeles/pharmAIcy>